YCCE

Vision

"To become the most preferred institution providing innovative, research and value based, professional education for the society at large".

Mission

YCCE is committed to

- Attract best talent and create learning ambience
- Practice Innovative teaching-learning & research
- Integrate Industry-Institute Collaborations
- Nurture students towards holistic development and choicest career

Department

Vision of the Department

To be a well-known center for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary arena by developing problem-solving skills through emerging technologies.

Nagar Yuwak Shikshan Sanstha's

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)

Hingna Road, Wanadongri, Nagpur - 441 110

Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Science and Engineering (IOT)

23CT1402	Lab: Operating Systems

Name of the Student: Sheikh Wasimuddin	Semester/ Section: 5 A
Roll No: 64	Enrollment Number: 23071044

Sr. No.	COs	POs												PSOs	
		Course Outcomes	1	2	3	4	5	6	7	8	9	10	11	PSO 1	PSO2
1	CO1	Demonstrate the ability to execute Linux process management, memory management, and shell commands to manage system resources efficiently.	3	3	3	-		-	-	-	-	-	3	3	
2	CO2	Develop programs utilizing system calls, thread programming, and page replacement algorithms to simulate and analyze operating system functionalities.	3	3	3	-	-	-	-	-	-	-	3	3	
3	CO3	Design and implement process scheduling, memory allocation, and deadlock detection algorithms to address real-world operating system challenges.	3	3	3	-		-	-	-	-	-	3	3	
		Avg		3	3	-		-	-	-	-	-	3	3	

Nagar Yuwak Shikshan Sanstha's

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)

Hingna Road, Wanadongri, Nagpur - 441 110

Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Science and Engineering (IOT)

Practical No. 2

Aim: Implement programs of process, file, and device management using appropriate Linux system calls.

Theory:

- **Fork ():** The fork () system call is used by processes to create copies of themselves. It is one of the methods used the most frequently in operating systems to create processes.

- **Wait ()**: In some systems, a process might need to hold off until another process has finished running before continuing. When a parent process creates a child process, the execution of the parent process is halted until the child process is complete.

- **Exit ():** A system call called exit () is used to terminate a program. In environments with multiple threads, this call indicates that the thread execution is finished.

- **Write ():** Data from a user buffer is written using it to a device like a file. A program can produce data in one way by using this system call. generally, there are three arguments.

  1. A description of a file.
  2. A reference to the buffer where data is stored.
  3. The amount of data that will be written from the buffer in bytes.

- **Open ():** Accessing a file on a file system is possible with the open() system call. It gives the file resources it needs and a handle the process can use. A file can be opened by multiple processes simultaneously or just one process. Everything is based on the structure and file system.

- **Read ():** Data from a file on the file system is retrieved using it. In general, it accepts three arguments:
  1. A description of a file.
  2. A buffer for read data storage.
  3. How many bytes should be read from the file.

- ***exec ():*** Replaces the current process's memory space with a new program, effectively executing a different program.

- **Seek ():** The seek () (or more precisely seek () in C) system call repositions the file offset (the pointer indicating the current read/write position) of an open file descriptor.
- **Close ():** The close () system call terminates the association between an open file descriptor and its file. Once closed, the file descriptor becomes available for reuse by the system.

Algorithm (If Applicable)

Code:

1.

```
//Sheikh Wasimuddin _64_CSEIot_practical-2_process system call program 1

#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main () {
   fork ();
   printf("Hello my name is sheikh Wasimuddin! \n");

   return 0;
}
```

2.

```
//Sheikh Wasimuddin _64_CSEIot_practical-2_process system call_program 2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        printf("Child: I am the child process\n");
    }
    else if (pid > 0) {
        wait(NULL);
        printf("Parent: Child has finished.\n");
    }
    else {
        printf("Fork failed!\n");
    }

    return 0;
}
```

3.

```
//Sheikh Wasimuddin _64_CSEIot_practical-2_file management_program 1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd;
    char buffer[100];
    fd = open("test.txt", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    read(fd, buffer, sizeof(buffer));
    printf("File content:\n%s\n", buffer);
    close(fd);
    if (rename("test.txt", "renamed.txt") == 0) {
        printf("File renamed successfully.\n");
    } else {
        perror("rename");
    }
    return 0;
}
```

OUTPUT (SCREEN SHOT) IF ANY:

main.c

```c
//Sheikh Wasimuddin _64_CSEIot_practical-2_process system call_program 1
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main(){
    fork();
    printf("Hello my name is sheikh Wasimuddin!\n");
    return 0;
}
```

```
Hello my name is sheikh Wasimuddin!
Hello my name is sheikh Wasimuddin!

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c

```c
//Sheikh Wasimuddin _64_CSEIot_practical-2_process system call_program 2
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        printf("Child: I am the child process\n");
    }
    else if (pid > 0) {
        wait(NULL);
        printf("Parent: Child has finished.\n");
    }
    else {
        printf("Fork failed!\n");
    }

    return 0;
}
```

```
Child: I am the child process
Parent: Child has finished.

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c | test.txt | renamed.txt

```c
//Sheikh Wasimuddin _64_CSEIot_practical-2_file management_program 1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main() {
    int fd;
    char buffer[100];
    fd = open("test.txt", O_RDONLY);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    read(fd, buffer, sizeof(buffer));
    printf("File content:\n%s\n", buffer);
    close(fd);
    if (rename("test.txt", "renamed.txt") == 0) {
        printf("File renamed successfully.\n");
    } else {
        perror("rename");
    }
    return 0;
}
```

```
File content:
my name is sheikh wasimuddin
File renamed successfully.

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c | test.txt | renamed.txt

```
my name is sheikh wasimuddin
```

```
File content:
my name is sheikh wasimuddin
File renamed successfully.

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c | test.txt | renamed.txt

```
my name is sheikh wasimuddin
```

```
File content:
my name is sheikh wasimuddin
File renamed successfully.

...Program finished with exit code 0
Press ENTER to exit console.
```

Sample Output

1.
Hello my name is sheikh Wasimuddin!
Hello my name is sheikh Wasimuddin!

2.
Child: I am the child process
Parent: Child has finished.

3.
File content:
My name is sheikh wasimuddin
File renamed successfully.

Conclusion: In this practical, various Linux system calls such as fork (), wait(), open(), read(), and rename () were implemented to demonstrate process creation, synchronization, and basic file management operations. This enhances understanding of process management and system resource handling in an operating system.